The Ordinary Product Recommender

Presenter View please!

Problem Statement

The skincare industry is vast and overwhelming, with numerous product options available in the market.

Customers often struggle to identify the most suitable skincare products that align with their individual characteristics and preferences. This lack of personalized guidance can lead to frustration and unsatisfactory product choices.

Inaccurate Value Record

In [60]:
```
# Count the number of data points where 'total_feedback_count' is 0
count_zero_feedback = df[df['total_feedback_count'] == 0].shape[0]

# Print the count
print("Number of data points with total_feedback_count = 0:", count_zero_feedback)
```

```
Number of data points with total_feedback_count = 0: 7620
```

In [61]:
```
# Count the value occurrences in the 'rating' column of the filtered DataFrame
rating_counts = df[df['total_feedback_count'] == 0]['rating'].value_counts()

# Print the value counts
print(rating_counts)
```

```
5    5326
4    1163
3     508
2     406
1     217
Name: rating, dtype: int64
```

'total_feedback_count'
Total number of feedback (positive and negative ratings) left by users for the review

~~First assumption:~~
~~Neutral ratings = 0 total feedback count~~

Data Cleaning

Before

```
In [5]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41658 entries, 0 to 41657
Data columns (total 17 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   author_id                 41658 non-null  object
 1   rating                    41658 non-null  int64
 2   is_recommended            41658 non-null  float64
 3   helpfulness               34038 non-null  float64
 4   total_feedback_count      41658 non-null  int64
 5   total_neg_feedback_count  41658 non-null  int64
 6   total_pos_feedback_count  41658 non-null  int64
 7   submission_time           41658 non-null  object
 8   review_text               41522 non-null  object
 9   review_title              28668 non-null  object
 10  skin_tone                 40585 non-null  object
 11  eye_color                 39936 non-null  object
 12  skin_type                 41116 non-null  object
 13  hair_color                39783 non-null  object
 14  product_id                41658 non-null  object
 15  product_name              41658 non-null  object
 16  price_usd                 41658 non-null  float64
dtypes: float64(3), int64(4), object(10)
memory usage: 5.4+ MB
```

After

```
In [79]: df.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 29303 entries, 0 to 41655
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   author_id                 29303 non-null  object
 1   rating                    29303 non-null  int64
 2   is_recommended            29303 non-null  float64
 3   helpfulness               29303 non-null  float64
 4   total_feedback_count      29303 non-null  int64
 5   total_neg_feedback_count  29303 non-null  int64
 6   total_pos_feedback_count  29303 non-null  int64
 7   submission_time           29303 non-null  datetime64[ns]
 8   skin_tone                 29303 non-null  object
 9   skin_type                 29303 non-null  object
 10  product_id                29303 non-null  int32
 11  product_name              29303 non-null  object
 12  price_usd                 29303 non-null  float64
dtypes: datetime64[ns](1), float64(3), int32(1), int64(4), object(4)
memory usage: 3.0+ MB
```

Result before Methods and Improvements :

```
# 1. Logistic Regression
logreg = LogisticRegression()
logreg.fit(X_train, y_train)
y_pred_logreg = logreg.predict(X_test)
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)
print("Logistic Regression Accuracy:", accuracy_logreg)
```

```
Logistic Regression Accuracy: 0.829507637170407
```

```
# 2. Decision Trees
decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train, y_train)
y_pred_dt = decision_tree.predict(X_test)
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("Decision Trees Accuracy:", accuracy_dt)
```

```
Decision Trees Accuracy: 0.952982336376824
```

```
# 3. Random Forest
random_forest = RandomForestClassifier()
random_forest.fit(X_train, y_train)
y_pred_rf = random_forest.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", accuracy_rf)
```

```
Random Forest Accuracy: 0.9641607645703558
```

```
# 4. Support Vector Machines (SVM)
svm = SVC()
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print("SVM Accuracy:", accuracy_svm)
```

```
SVM Accuracy: 0.829507637170407
```

```
# 5. Naive Bayes
naive_bayes = GaussianNB()
naive_bayes.fit(X_train, y_train)
y_pred_nb = naive_bayes.predict(X_test)
accuracy_nb = accuracy_score(y_test, y_pred_nb)
print("Naive Bayes Accuracy:", accuracy_nb)
```

```
Naive Bayes Accuracy: 0.9612594931308132
```

```
# 6. K-Nearest Neighbors (KNN)
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("K-Nearest Neighbors Accuracy:", accuracy_knn)
```

```
K-Nearest Neighbors Accuracy: 0.81064937281338
```

```
# 7. Gradient Boosting algorithms
lightgbm = LGBMClassifier()
lightgbm.fit(X_train, y_train)
y_pred_lgbm = lightgbm.predict(X_test)
accuracy_lgbm = accuracy_score(y_test, y_pred_lgbm)
print("LightGBM Accuracy:", accuracy_lgbm)
```

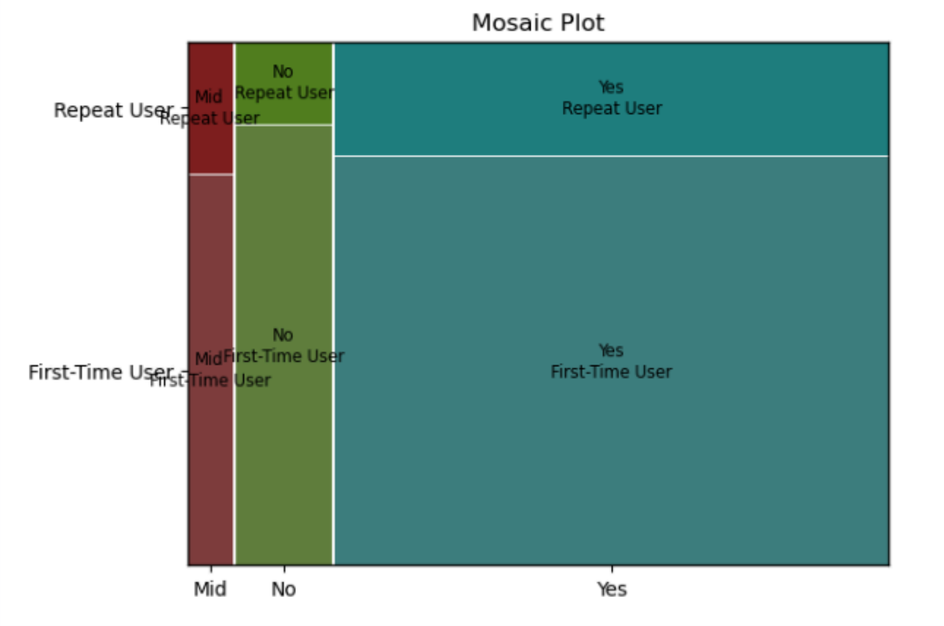
```
LightGBM Accuracy: 0.9630514549022954
```

Data Visualisation

Data Dictionary

Column Name	Level of Measurement
is_recommended	Nominal
helpfulness	Ordinal
skin_tone	Nominal
product_id	Nominal
product_name	Nominal
price_usd	Ratio
helpfulness_bin	Ordinal
author_bin_Repear User	Nominal
skin_tone_encoded	Nominal
skin_tone_bin_encoded	Nominal
helpfulness_encoded	Nominal
skin_type	Nominal
skin_tone_bin	Nominal

Findings



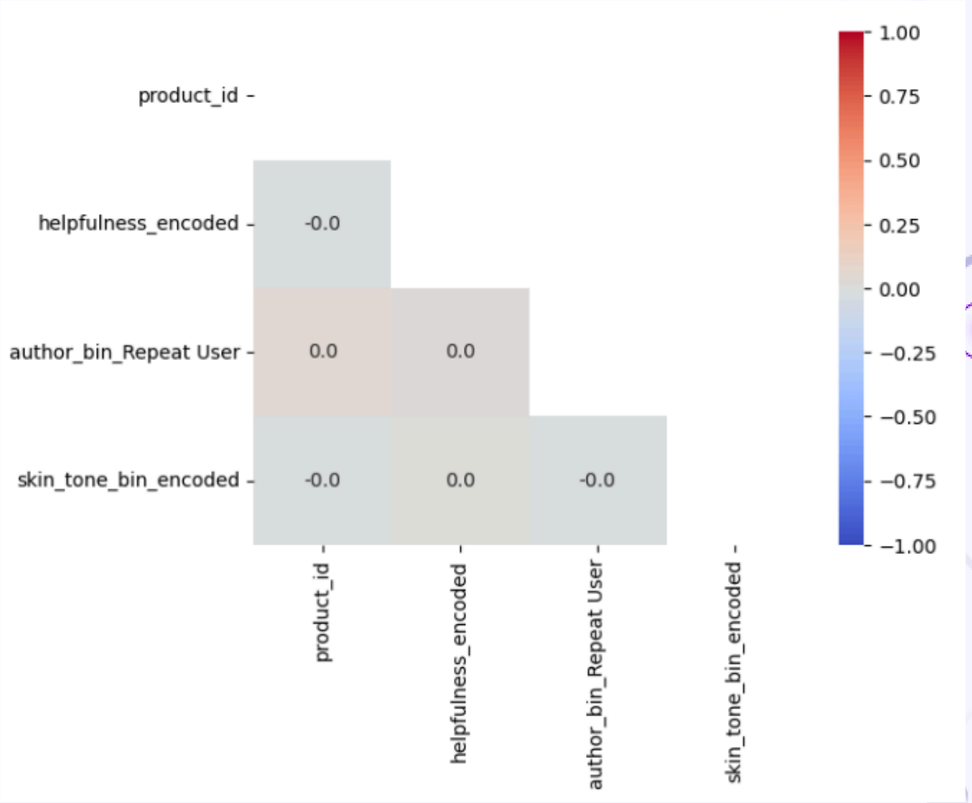
Repeat users tend to provide a higher proportion of positive and neutral reviews compared to first-time users.

This suggest that repeat users might have a stronger inclination towards liking our product, as indicated by their favorable review distribution.

Reviewing Features & Write out Datasets

Reduced Features

- No linear relationship or dependency between any pair of features.
- Having zero correlation can be advantageous as it indicates independence and uniqueness of information.



Evaluate Models on the Test Set

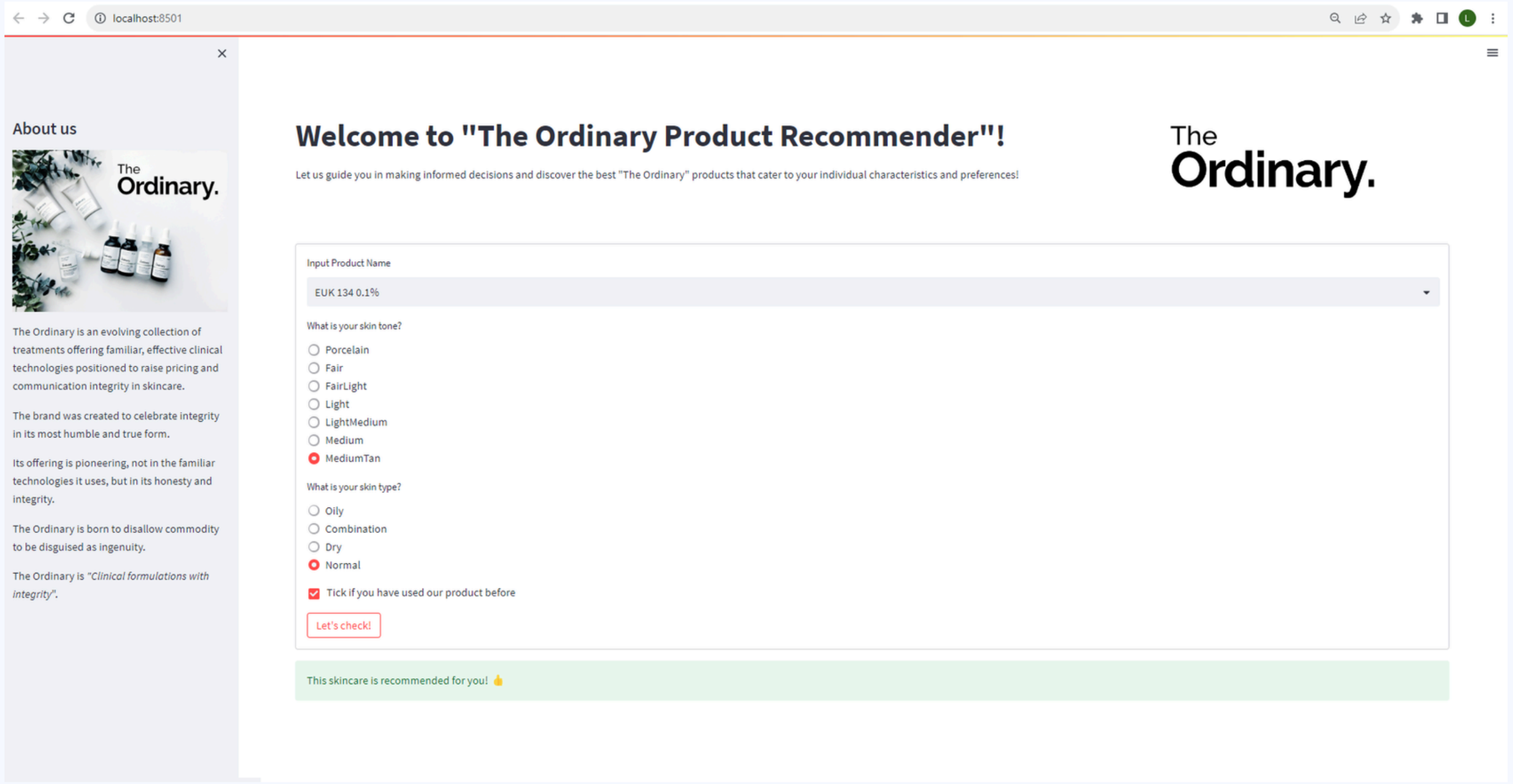
```
evaluate_model("All Features", all_model, test_features_all, test_labels)
```

```
All Features --         Accuracy: 0.971 / Precision: 0.996 / Recall: 0.968 / True Negatives: 836 / Latency: 1690655729ms
```

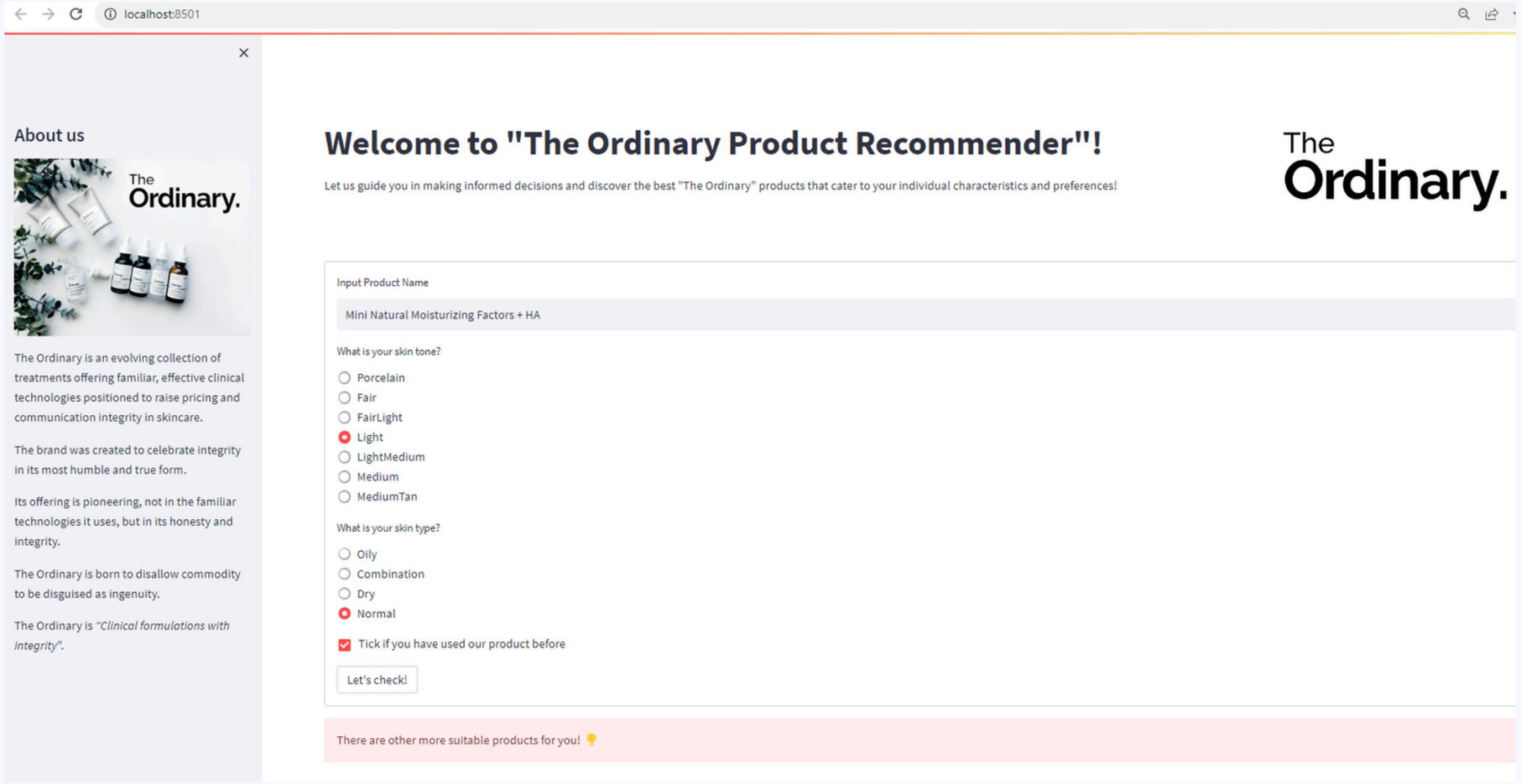
After a thorough evaluation of various models, using All features with Decision Tree machine learning algorithm demonstrated superior performance, surpassing all previous results in terms of accuracy, precision, and recall rates.

Although there was a slight decrease in the True Negative rate, the overall performance of this model is outstanding. Therefore, I have decided to use this high-performing model for the website deployment.

Website Deployment



Website Deployment



This is the end of the persentation.

Thank you!